

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

Nombre: Alejandro Mogollón Rodríguez

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH

Verifica Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo.

Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`

```
C:\url-mia-2026>cd Semana_08

C:\url-mia-2026\Semana_08>mkdir lab8

C:\url-mia-2026\Semana_08>cd lab8

C:\url-mia-2026\Semana_08\lab8>git --version
git version 2.49.0.windows.1

C:\url-mia-2026\Semana_08\lab8>ssh -T git@github.com
Hi amogollon1! You've successfully authenticated, but GitHub does not provide shell access.
```

Entra al repositorio y crea tu rama: `git checkout -b lab-[nombre_branch]`

```
C:\url-mia-2026\Semana_08\lab8>git checkout -b lab-8-Alejandro-Mogollon
Switched to a new branch 'lab-8-Alejandro-Mogollon'
```

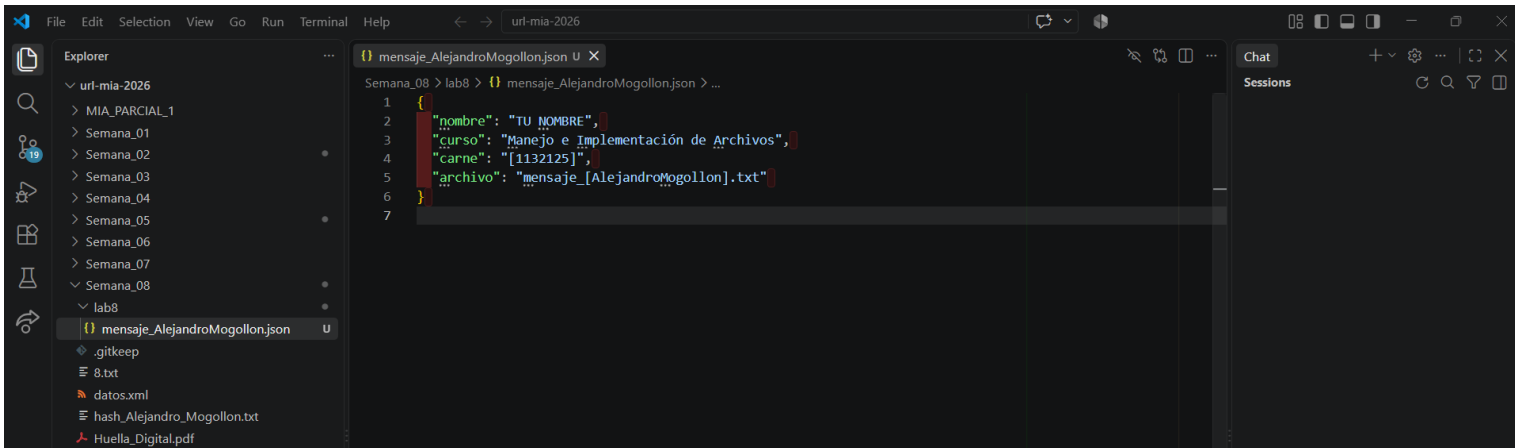
2. Crear un archivo personalizado

Creas un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: `mensaje_LuisRojas.json`

Contenido:

```
{
  "nombre": "TU NOMBRE",
  "curso": "Manejo e Implementación de Archivos",
  "carne": "[tu_carne]",
  "archivo": "mensaje_[Nombre_apellido].txt"
}
```



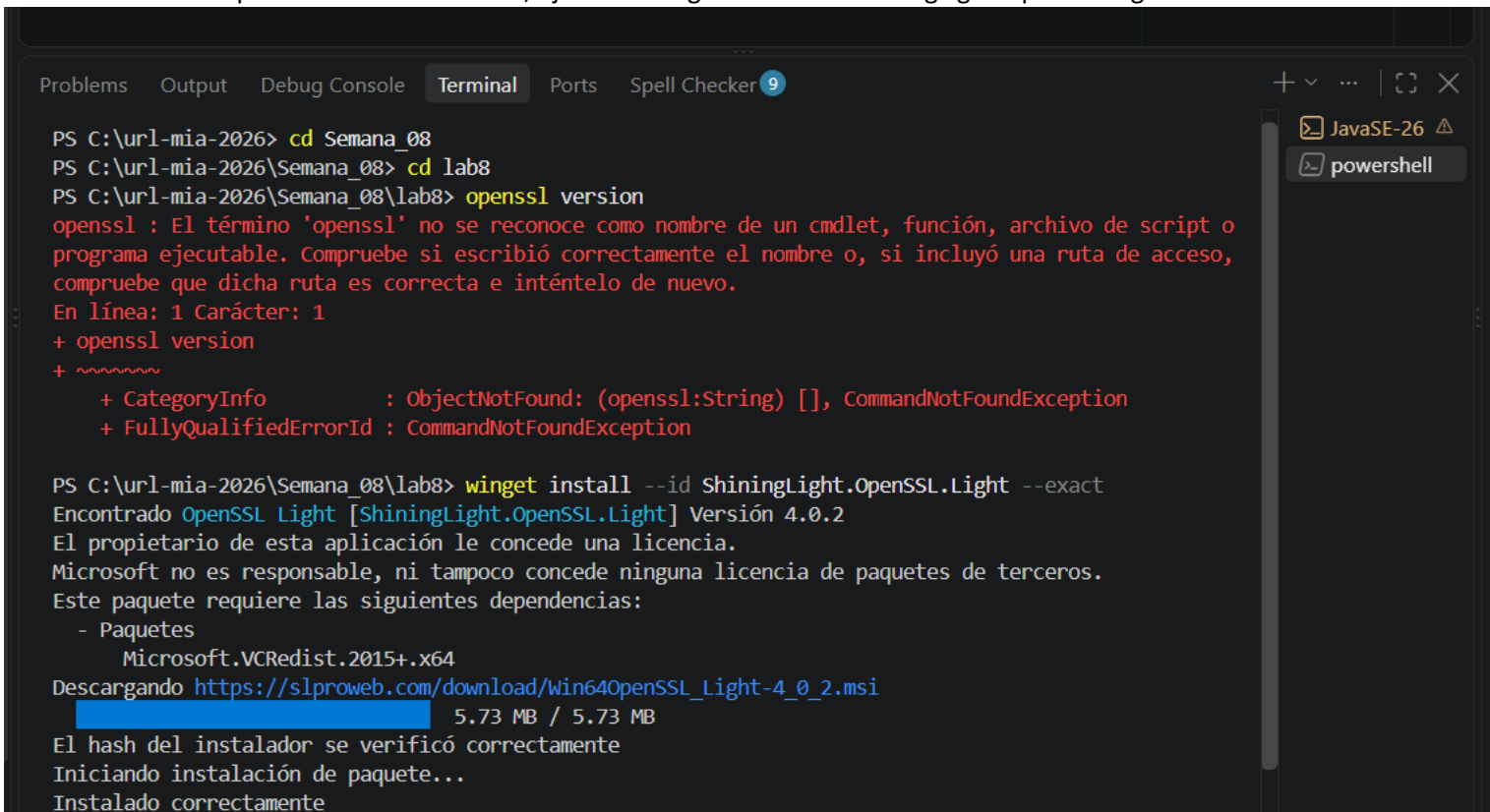
3. Encriptar y cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión **.enc**

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact`



Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"

3.3 Encriptar tu archivo

Comprueba dónde quedó instalado: Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"

Agrega OpenSSL al PATH de tu usuario: \$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"

[Environment]::SetEnvironmentVariable("Path", \$env:Path + ";" + \$opensslPath, "User")

Actualiza el PATH de esta terminal inmediatamente: \$env:Path += ";C:\Program Files\OpenSSLWin64\bin"

```
PS C:\url-mia-2026> cd Semana_08
PS C:\url-mia-2026\Semana_08> cd lab8
PS C:\url-mia-2026\Semana_08\lab8> openssl version
OpenSSL 4.0.2 25 Aug 2026 (Library: OpenSSL 4.0.2 25 Aug 2026)
PS C:\url-mia-2026\Semana_08\lab8> Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"
```

Directorio: C:\Program Files\OpenSSL-Win64\bin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	25/08/2026 09:21	821760	openssl.exe

```
PS C:\url-mia-2026\Semana_08\lab8> █
```

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

```
PS C:\url-mia-2026\Semana_08\lab8> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_AlejandroMogollon.json -out .\mensaje_AlejandroMogollon.enc
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

PS C:\url-mia-2026\Semana_08\lab8> █
```

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre`

```
PS C:\url-mia-2026\Semana_08\lab8> Get-ChildItem

Directorio: C:\url-mia-2026\Semana_08\lab8

Mode                LastWriteTime         Length Name
----                -
-a----            28/08/2026    09:23           176 mensaje_AlejandroMogollon.enc
-a----            28/08/2026    08:52           157 mensaje_AlejandroMogollon.json

PS C:\url-mia-2026\Semana_08\lab8> █
```

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc` ¿Qué observaste?

Se ve un conjunto de caracteres aleatorios que representan el contenido del archivo.

```
PS C:\url-mia-2026\Semana_08\lab8> Get-Content .\mensaje_AlejandroMogollon.enc
Salted__TX'~qL¥)!öxFT³q>vZÜo!ÄLiäi;Cöx8"mÄÖmx¿Æ^ÉAid¿ýæd†^ŸŽyif%om"ÍÉ:-,  iñóæI|B-J•i <w-,,»ÖÍóÊDİ6qtRĚ...?†"Ök|kĒBä9†=QÄÿSçÜ†~Žix2Ÿ-Š
PS C:\url-mia-2026\Semana_08\lab8> █
```

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

```
openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out  
.\mensaje_TuNombre_recuperado.json
```

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

```
PS C:\url-mia-2026\Semana_08\lab8> openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_AlejandroMogollon.enc -out .\mensaje_Alejan  
roMogollon_recuperado.json  
enter AES-256-CBC decryption password:  
  
PS C:\url-mia-2026\Semana_08\lab8> Get-Content .\mensaje_AlejandroMogollon_recuperado.json  
{  
  "nombre": "TU NOMBRE",  
  "curso": "Manejo e Implementaci³n de Archivos",  
  "carne": "[1132125]",  
  "archivo": "mensaje_[AlejandroMogollon].txt"  
}
```

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETa" .\mensaje_LuisRojas.json` ¿Cuál es el HMAC-SHA256 obtenido?

8e8b739fb23caedd9f0089a7d22d7f856f45526702cb85da863c91bf474ca5e3

```
PS C:\url-mia-2026\Semana_08\lab8> openssl dgst -sha256 -hmac "TU_CLAVE_SECRETa" .\mensaje_AlejandroMogollon.json  
HMAC-SHA2-256(. \mensaje_AlejandroMogollon.json)= 8e8b739fb23caedd9f0089a7d22d7f856f45526702cb85da863c91bf474ca5e3
```

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?

La confidencialidad, el contenido permanece privado e ilegible para usuarios no autorizados.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

Porque daría acceso al cifrado a cualquier persona que lo vea desde el repositorio, pudiendo así acceder al mensaje.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

SHA256 solo verifica que el archivo siga siendo el mismo, que no haya sido modificado.

HMAC-SHA256 requiere de una clave secreta para acceder, asegurando la integridad del archivo.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado.

Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.